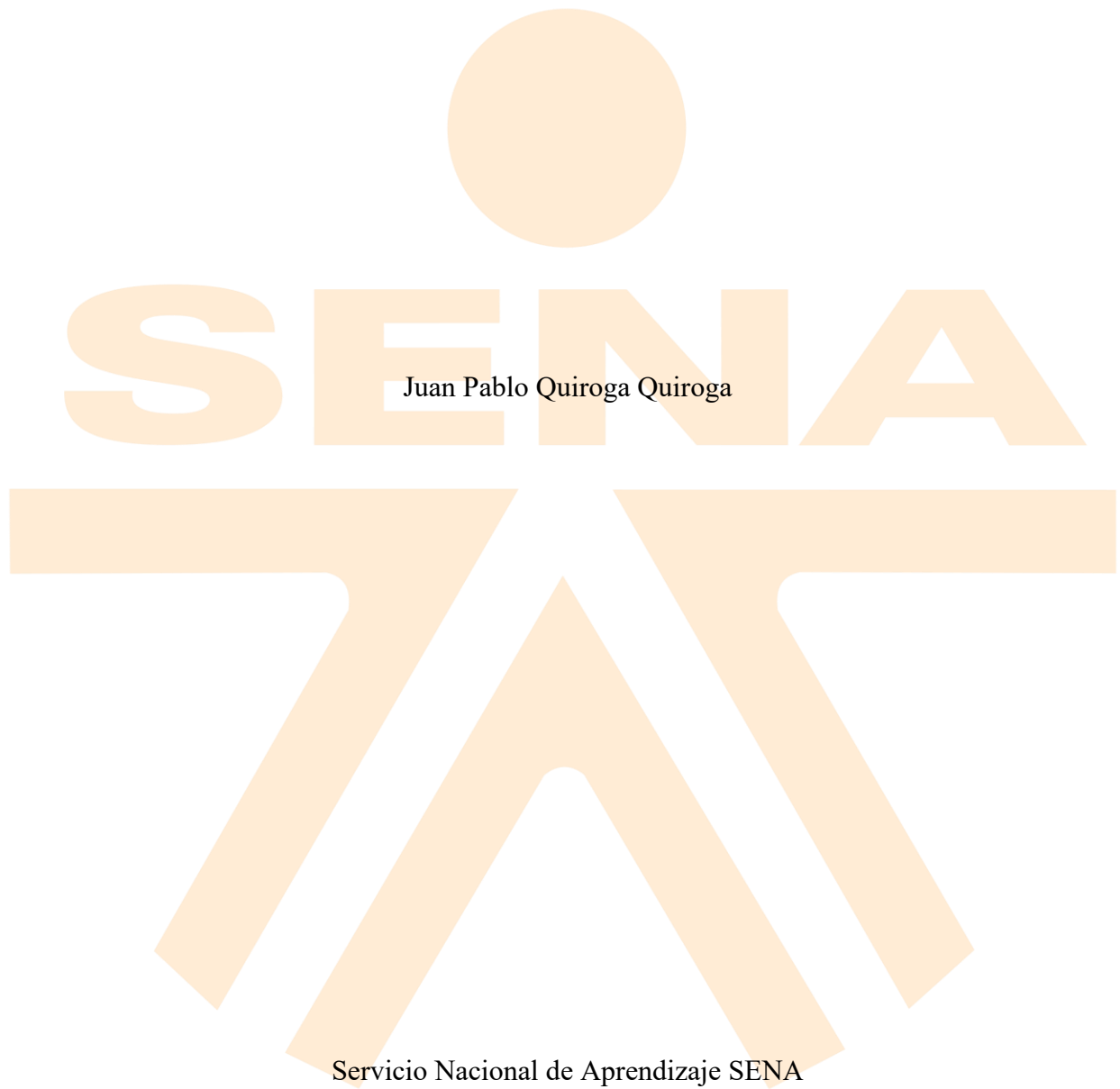


Desarrollar Módulos Móviles Según Requerimientos del Proyecto



Tecnólogo en Análisis y Desarrollo de Software Ficha – 3118562

Instructor: Henry Alfonso Garzón Sanchez

Contenido

Introducción	5
Sección 1	6
Configuración del Proyecto	6
Estructura del Proyecto	6
Base de Datos.....	7
Paleta de Colores.....	8
Navegación	10
Pantalla de Login	11
Dashboard	12
Nacionalidades.....	14
Registrar Docente.....	15
Formulario de Datos del Docente	17
Entidades.....	18
Nacionalidad	18
Docente	19
Persona.....	19
Rol.....	19
Usuario.....	20

Base de Datos.....	20
Acceso a la Base de Datos	21
DocenteDao.....	21
NacionalidadDao.....	22
PersonaDao	23
RolDao	24
UsuarioDao	25
ViewModels.....	26
DocenteViewModel	26
LoginViewModel.....	27
NacionalidadViewModel	28
PersonaViewModel.....	29
Sección 2.....	30
Constraint Layout.....	30
Codificación del Layout.....	31
Login con Linear Layout (Column).....	31
Login con Constraint Layout.	31
Integración entre ambas versiones	32
Validación de Usuario con SQLite y Android Studio	35
Paso 1 Encriptación de la contraseña.....	35

Paso 2 Consulta a la base de datos..... 35

Paso 3 Respuesta de la app. 36



Introducción

El presente documento presenta el desarrollo de la evidencia Desarrollar Módulos Móviles Según Requerimientos del Proyecto.

Para esta evidencia se desarrolló una aplicación móvil Android para la gestión de docentes. La app permite al administrador del sistema iniciar sesión, registrar nacionalidades y registrar nuevos docentes con sus datos personales y laborales, almacenando toda la información de forma local en el dispositivo.

La aplicación fue construida con Kotlin como lenguaje de programación y Jetpack Compose como framework de interfaz de usuario — la tecnología moderna recomendada por Google para el desarrollo de apps Android. Para el almacenamiento de datos se utilizó Room, la librería oficial de Android para el manejo de bases de datos SQLite, y la arquitectura MVVM (Model-View-ViewModel) para organizar el código de forma clara y mantenible.

Como parte del ejercicio se implementaron dos tipos de layout en la pantalla de autenticación: Linear Layout, usando Column de Jetpack Compose para organizar los elementos en columna vertical, y Constraint Layout, donde cada elemento se posiciona mediante restricciones de anclaje, demostrando las diferencias entre ambos enfoques de diseño de interfaces. Adicionalmente se documentó el proceso de validación de usuario contra la base de datos SQLite local.

Sección 1

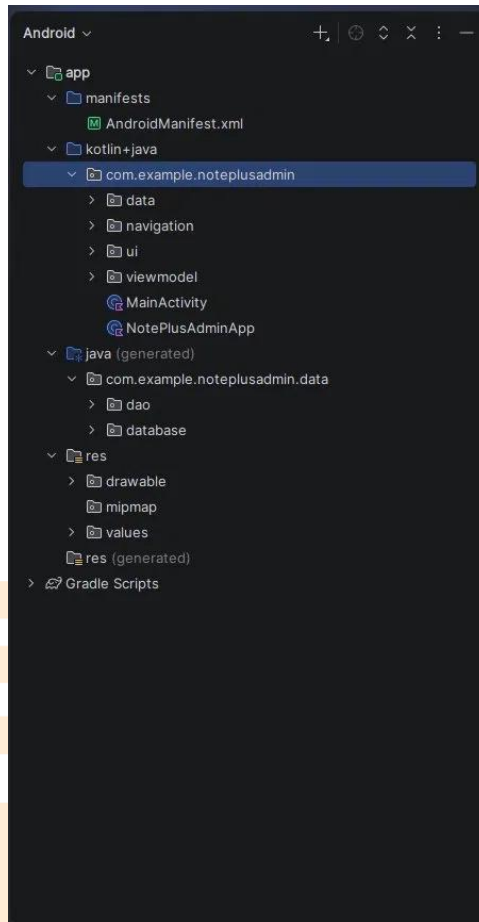
Configuración del Proyecto

Estructura del Proyecto

Esta evidencia fue creada en Android Studio utilizando Kotlin como lenguaje de programación. La aplicación está organizada en las siguientes carpetas que indican la responsabilidad de cada parte:

- **data** — contiene todo lo relacionado con la base de datos: las tablas (entidades), los objetos de acceso a datos (DAOs) y la configuración de la base de datos Room
- **navigation** — contiene la configuración de la navegación entre pantallas
- **ui** — contiene todas las pantallas visibles de la aplicación
- **viewmodel** — contiene la lógica de cada pantalla, conectando la interfaz con la base de datos
- **MainActivity** — es el punto de entrada de la aplicación, la primera clase que se ejecuta al abrir la app
- **NotePlusAdminApp** — clase de configuración general de la aplicación
- **res** — contiene los recursos visuales como íconos y colores
- **AndroidManifest.xml** — es el archivo de configuración general de la app donde se declaran los permisos y la actividad principal

Esta estructura sigue el patrón de arquitectura **MVVM** (Model-View-ViewModel), que separa claramente los datos, la lógica y la interfaz de usuario.



Base de Datos

Para el almacenamiento de los datos se utilizó **Room**, que es la librería oficial de Android para manejar bases de datos **SQLite** de forma sencilla. La carpeta data está dividida en tres subcarpetas:

- **entities** — contiene las 5 tablas de la base de datos representadas como clases Kotlin:
 - Nacionalidad — almacena los tipos de nacionalidad disponibles
 - Persona — almacena los datos personales de cada docente registrado
 - Rol — define los roles del sistema
 - Usuario — almacena las credenciales de acceso al sistema

- Docente — almacena los datos laborales del docente, vinculado a una Persona
- **dao** — contiene las clases que definen las operaciones disponibles sobre cada tabla: insertar registros, consultarlos y buscar por credenciales para el login
- **database** — contiene la configuración principal de la base de datos, incluyendo la carga automática de datos iniciales al instalar la app por primera vez (usuario admin, nacionalidades y roles predefinidos)

Paleta de Colores

Paleta de Colores

Los colores de la aplicación se definieron en el archivo `Color.kt` dentro de la carpeta `ui/theme/`. Cada color tiene un nombre descriptivo que indica su uso dentro de la app, organizados en cuatro grupos:

Verdes principales:

- `GreenPrimary (#16a34a)` — usado en botones principales y acentos, aparece 31 veces en la app
- `GreenDark (#15803d)` — usado en los headers y barras de navegación superiores
- `GreenLight (#dcfce7)` — usado en fondos suaves y tarjetas
- `GreenOnPrimary (ffffff)` — texto blanco sobre fondos verdes

Fondos:

- `White (ffffff)` — fondo general en modo claro
- `DarkBackground (#1f2937)` — fondo en modo oscuro

- DarkSurface (#374151) — superficies en modo oscuro

Semánticos:

- ErrorRed (#dc2626) — mensajes de error y botones de eliminar
- WarningYellow (#eab308) — mensajes de advertencia
- InfoBlue (#0ea5e9) — mensajes informativos

Neutros:

- TextPrimary y TextSecondary — colores de texto principal y secundario
- Divider — líneas separadoras entre elementos

```
Color.kt
1 package com.example.noteplusadmin.ui.theme
2
3 import androidx.compose.ui.graphics.Color
4
5 // Verdes principales
6 31 Usages
7 val GreenPrimary = Color(0xFF16a34a)
8 22 Usages
9 val GreenDark = Color(0xFF15803d)
10 9 Usages
11 val GreenLight = Color(0xFFdcfce7)
12 1 Usage
13 val GreenOnPrimary = Color(0xFFFFFFFF)
14
15 // Fondos
16 28 Usages
17 val White = Color(0xFFFFFFFF)
18 1 Usage
19 val DarkBackground = Color(0xFF1f2937)
20 1 Usage
21 val DarkSurface = Color(0xFF374151)
22
23 // Semánticos
24 10 Usages
25 val ErrorRed = Color(0xFFdc2626)
26 val WarningYellow = Color(0xFFeab308)
27 val InfoBlue = Color(0xFF0ea5e9)
28
29 // Neutros
30 7 Usages
31 val TextPrimary = Color(0xFF111827)
32 8 Usages
33 val TextSecondary = Color(0xFF6b7280)
34 1 Usage
35 val Divider = Color(0xFFe5e7eb)
36
```

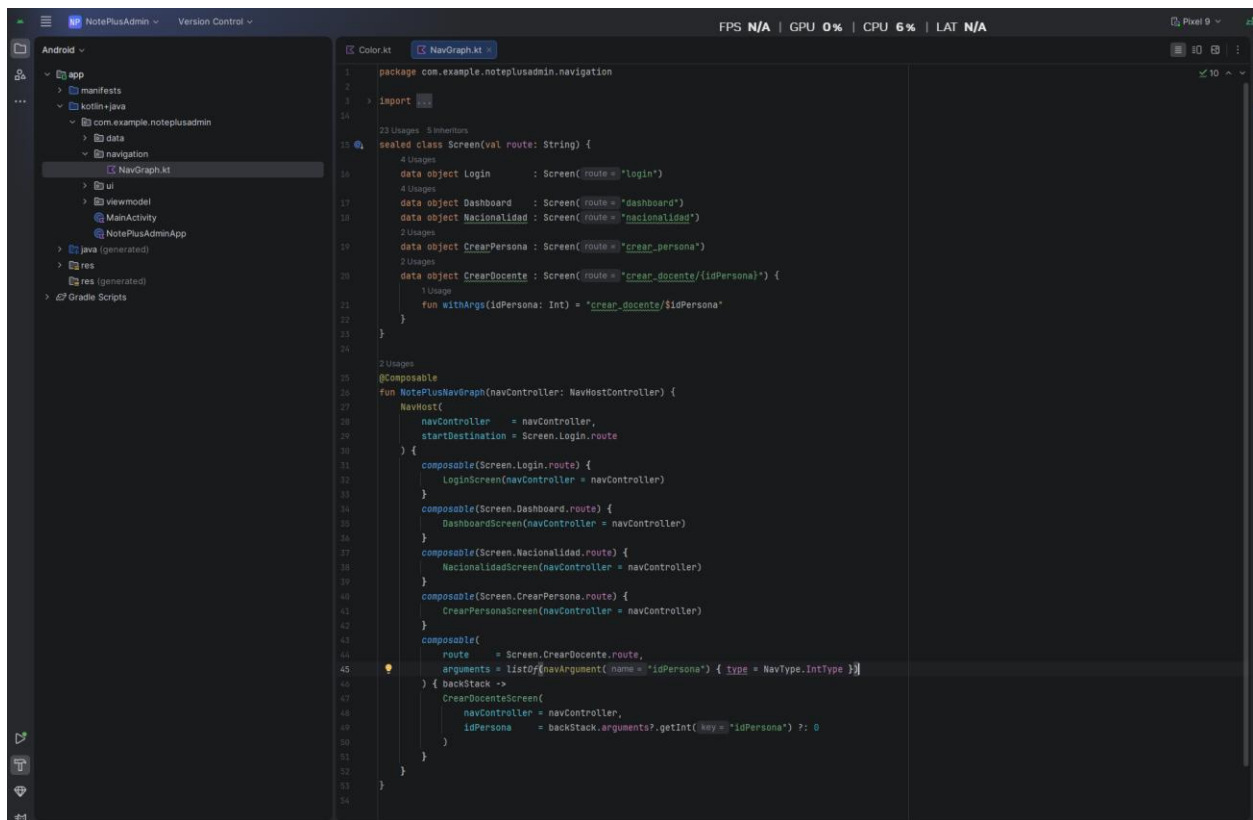
Navegación

La navegación de la app se configuró en el archivo NavGraph.kt dentro de la carpeta navigation/. En este archivo se definen todas las rutas disponibles y cómo se conectan entre sí.

Se definieron 5 pantallas usando una clase llamada Screen, donde cada una tiene su ruta:

- login — pantalla de inicio de sesión, es la primera que ve el usuario al abrir la app
- dashboard — menú principal después de iniciar sesión
- nacionalidad — pantalla para gestionar nacionalidades
- crear_persona — formulario de datos personales del docente
- crear_docente/{idPersona} — formulario de datos laborales del docente, recibe el ID de la persona recién creada para vincularlos

La función NotePlusNavGraph conecta todas las pantallas indicando que la app siempre arranca en el Login. Cuando el usuario navega de CrearPersona a CrearDocente, el ID de la persona se pasa automáticamente entre pantallas para que al guardar el docente quede correctamente vinculado a su información personal.



Pantalla de Login

La pantalla de inicio de sesión es lo primero que ve el usuario al abrir la aplicación. Se diseñó con fondo blanco y los elementos de la paleta de colores verde definida para el proyecto.

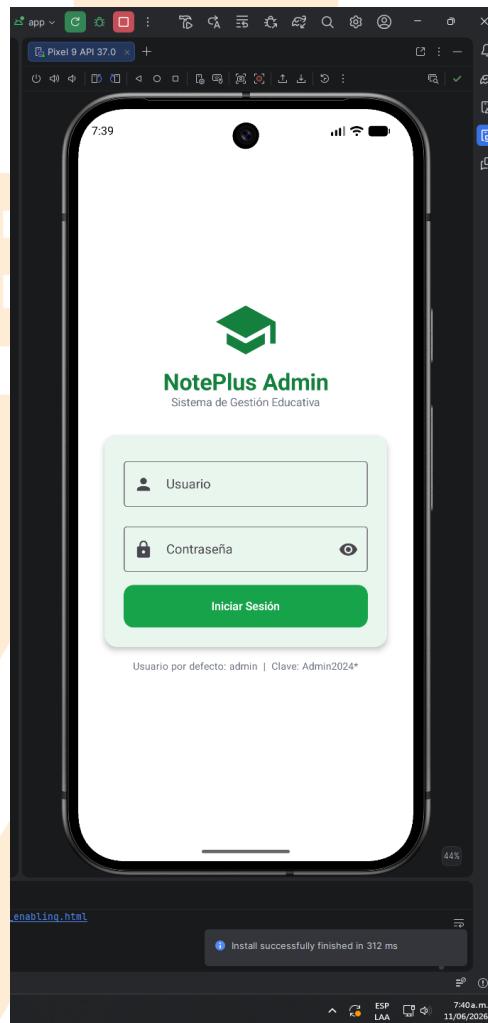
En la parte superior se muestra el ícono de la aplicación seguido del nombre **"NotePlus Admin"** y el subtítulo **"Sistema de Gestión Educativa"**.

El formulario de acceso contiene:

- **Campo Usuario** — con ícono de persona a la izquierda
- **Campo Contraseña** — con ícono de candado y un botón de ojo a la derecha que permite mostrar u ocultar la contraseña
- **Botón "Iniciar Sesión"** — en verde principal con texto blanco

Al ingresar las credenciales correctas (admin / Admin2026*) el sistema valida contra la base de datos local, convierte la contraseña a su forma encriptada y la compara con la almacenada. Si son correctas, navega al Dashboard. Si son incorrectas, muestra un mensaje de error en rojo.

En la parte inferior se muestra un recordatorio con las credenciales por defecto para facilitar el acceso durante las pruebas.



Dashboard

Después de iniciar sesión correctamente, el usuario llega al Panel de Administración. La pantalla tiene tres secciones principales:

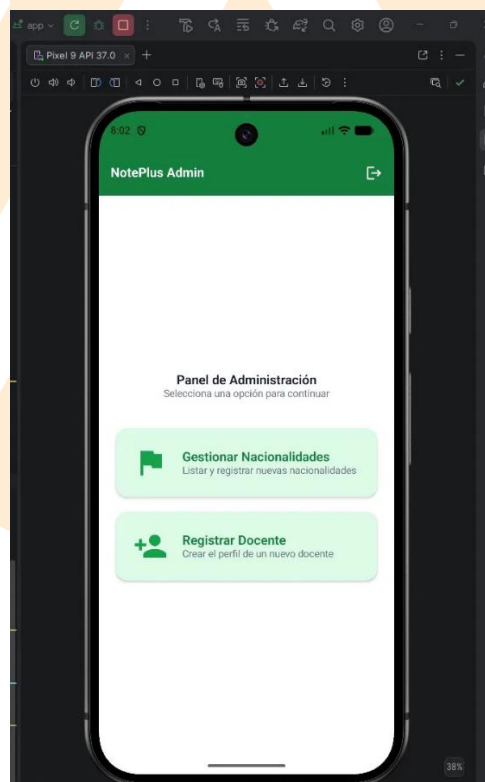
Barra superior: muestra el nombre de la aplicación **"NotePlus Admin"** en blanco sobre fondo verde oscuro. En la esquina derecha hay un botón de cierre de sesión representado con un ícono de salida — al pulsarlo el usuario regresa a la pantalla de Login.

Título central: muestra el texto **"Panel de Administración"** con el subtítulo **"Selecciona una opción para continuar"**, orientando al usuario sobre lo que puede hacer.

Tarjetas de navegación: dos opciones presentadas como tarjetas con fondo verde claro:

- **Gestionar Nacionalidades** — con ícono de bandera verde, permite listar y registrar nuevas nacionalidades en el sistema
- **Registrar Docente** — con ícono de persona verde, permite crear el perfil completo de un nuevo docente

Al hacer click en cualquiera de las tarjetas el sistema navega a la pantalla correspondiente.



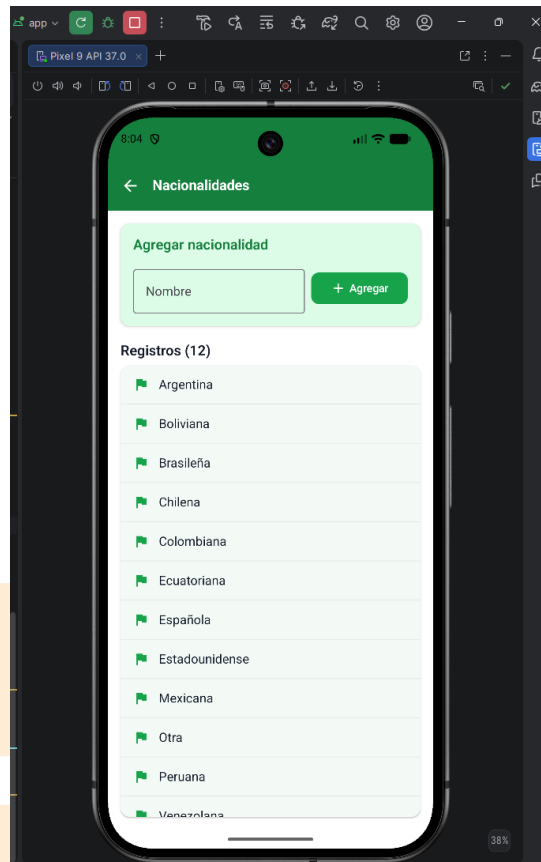
Nacionalidades

La pantalla de Nacionalidades permite ver y agregar nacionalidades al sistema. Está dividida en dos secciones:

Formulario para agregar: en la parte superior hay una tarjeta con fondo verde claro que contiene un campo de texto con el placeholder "Nombre" y un botón verde "+ **Agregar**". Al escribir el nombre de una nueva nacionalidad y pulsar el botón, se guarda inmediatamente en la base de datos y aparece en la lista. Si se intenta agregar una nacionalidad que ya existe, el sistema muestra un mensaje de error indicando que ya está registrada.

Lista de registros: debajo del formulario aparece el título "**Registros (12)**" indicando cuántas nacionalidades hay en total. Cada nacionalidad se muestra en una lista con un ícono de bandera verde a la izquierda y el nombre a la derecha, ordenadas alfabéticamente. Los 12 registros iniciales fueron insertados automáticamente al instalar la app por primera vez.

La barra superior muestra el título "**Nacionalidades**" con un botón de flecha hacia atrás para regresar al Dashboard.



Registrar Docente

Esta es la primera de las dos pantallas para registrar un docente. Como un docente es también una persona, primero se capturan todos sus datos personales antes de pasar a los datos laborales.

El formulario es largo y se puede desplazar hacia abajo. Está organizado en tres secciones:

Información del documento:

- Tipo de documento — dropdown con opciones CC, CE y Pasaporte
- Número de documento — campo de texto libre

Nombres y apellidos:

- Primer nombre — obligatorio
- Segundo nombre — opcional
- Primer apellido — obligatorio
- Segundo apellido — opcional

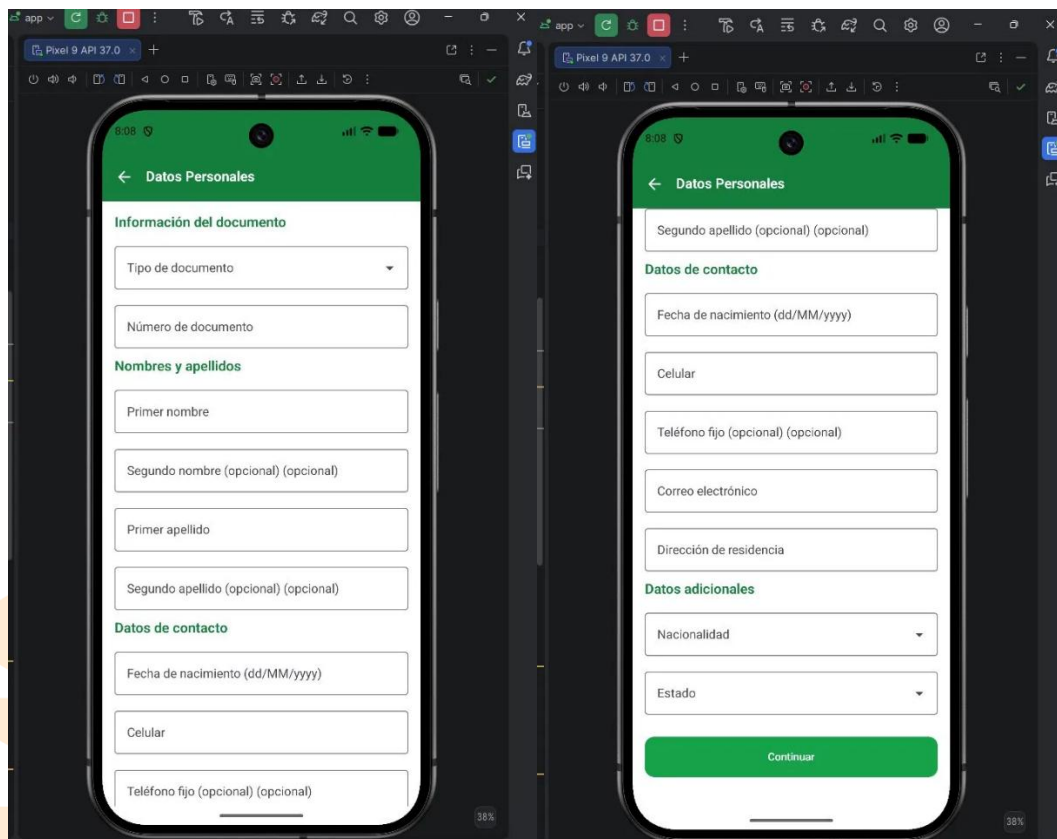
Datos de contacto:

- Fecha de nacimiento — campo con formato dd/MM/yyyy
- Celular — obligatorio
- Teléfono fijo — opcional
- Correo electrónico — obligatorio
- Dirección de residencia — obligatorio

Datos adicionales:

- Nacionalidad — dropdown cargado automáticamente desde la base de datos con todas las nacionalidades registradas
- Estado — dropdown con opciones Activo e Inactivo

Al final aparece el botón "**Continuar**" en verde. Al pulsarlo el sistema valida que todos los campos obligatorios estén completos, guarda la persona en la base de datos y navega automáticamente a la siguiente pantalla pasando el ID de la persona recién creada.



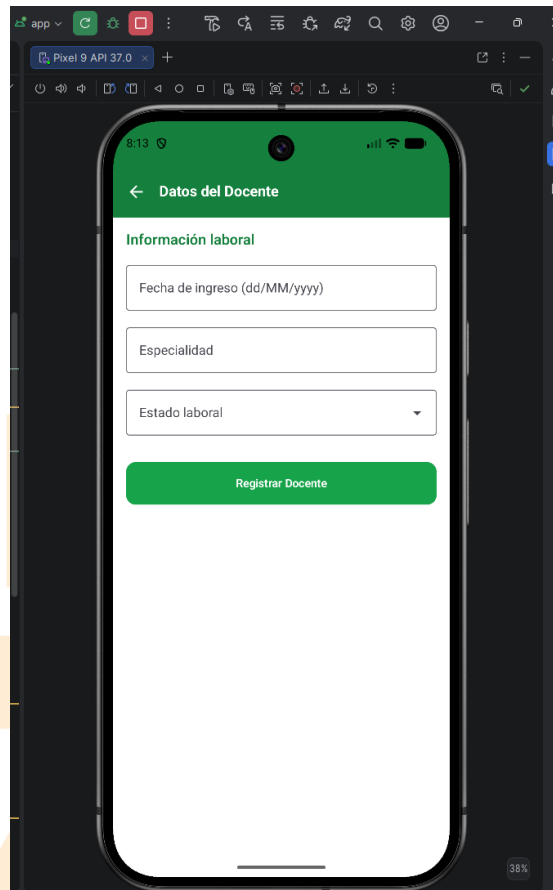
Formulario de Datos del Docente

Esta es la segunda y última pantalla del proceso de registro. Aparece automáticamente después de guardar los datos personales, recibiendo en segundo plano el ID de la persona recién creada para vincularlo correctamente.

El formulario contiene la sección "**Información laboral**" con tres campos:

- **Fecha de ingreso** — campo de texto con formato dd/MM/yyyy indicando cuándo el docente empezó a laborar
- **Especialidad** — campo de texto libre donde se escribe el área de conocimiento del docente
- **Estado laboral** — dropdown con las opciones Activo, Inactivo y Suspendido

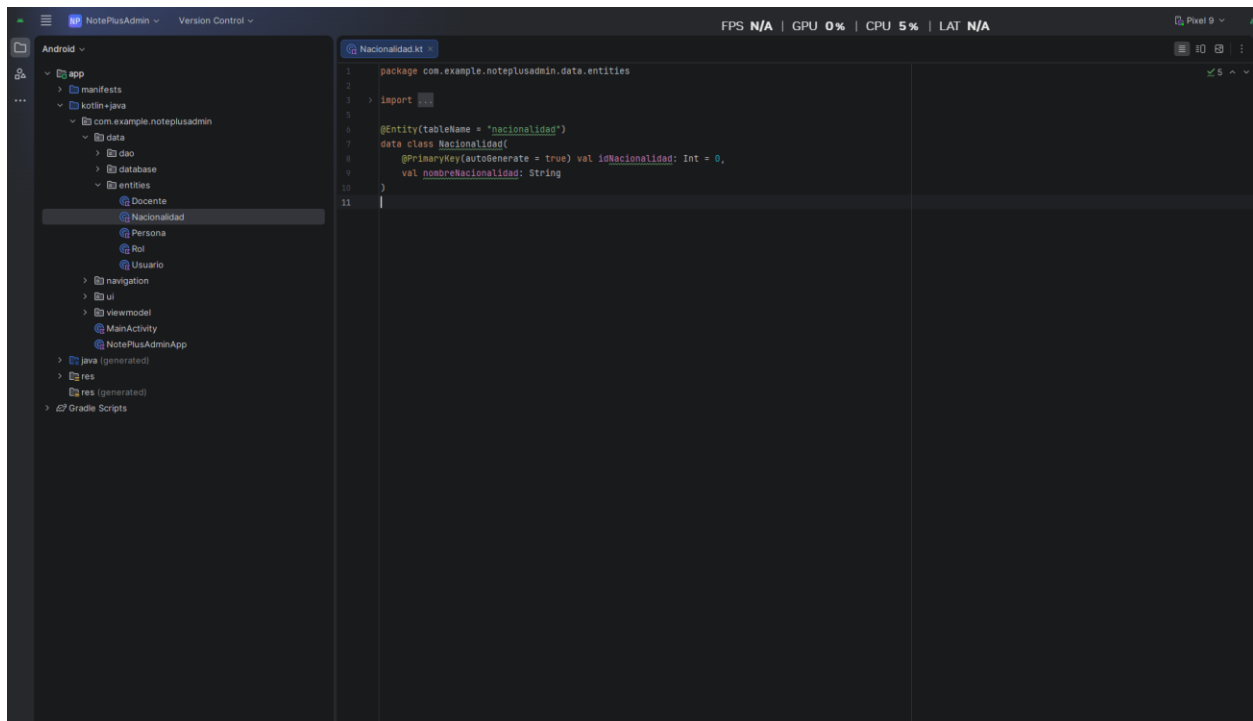
Al pulsar el botón "**Registrar Docente**" en verde, el sistema guarda el registro en la tabla Docente de la base de datos vinculándolo a la persona.

The image shows a mobile application interface for registering a teacher. The screen is titled "Datos del Docente" (Teacher Data) and features a green header bar. Below the header, there is a section titled "Información laboral" (Labor Information). This section contains three input fields: "Fecha de ingreso (dd/MM/yyyy)" (Entry date), "Especialidad" (Specialty), and "Estado laboral" (Labor status), which is a dropdown menu. At the bottom of this section is a prominent green button labeled "Registrar Docente" (Register Teacher). The entire interface is displayed within a browser window, with the address bar showing "Pixel 9 API 37.0".

Entidades

Nacionalidad

El archivo Nacionalidad.kt define la tabla nacionalidad en la base de datos. Tiene dos campos: el ID que se genera automáticamente y el nombre de la nacionalidad. Esta tabla funciona como un catálogo que alimenta la lista desplegable de nacionalidades en el formulario de registro de docentes.



Docente

Define la tabla docente con los datos laborales del docente: fecha de ingreso, especialidad y estado laboral. Está vinculada a la tabla Persona mediante una llave foránea — esto significa que no puede existir un docente sin una persona asociada. Si la persona se elimina, el registro del docente también se elimina automáticamente.

Persona

Es la tabla central del sistema. Almacena todos los datos personales: documento, nombres, apellidos, contacto y dirección. Está vinculada a la tabla Nacionalidad mediante llave foránea. Los campos de segundo nombre, segundo apellido y teléfono fijo son opcionales (marcados con ?). No se puede eliminar una nacionalidad si tiene personas asociadas.

Rol

Define la tabla rol con el nombre y descripción de cada rol del sistema. Es una tabla de catálogo y sus datos son fijos y se insertan automáticamente al instalar la app. Tiene dos campos opcionales: nombre y descripción.

Usuario

Define la tabla usuario con las credenciales de acceso al sistema. Está vinculada tanto a Rol como a Persona mediante llaves foráneas. El campo `contrasenaHash` almacena la contraseña, nunca en texto plano. Solo existe un usuario en el sistema que es el administrador insertado automáticamente al instalar la app.

Base de Datos

El archivo `AppDatabase.kt` es el núcleo del sistema de datos de la app. Cumple tres funciones principales:

1. Registro de tablas La anotación `@Database` le indica a Room cuáles son las 5 tablas de la app: Nacionalidad, Persona, Rol, Usuario y Docente. También define la versión de la base de datos (versión 1).

2. Instancia única (Singleton) La función `getDatabase()` garantiza que solo exista una conexión a la base de datos en toda la app. Esto evita conflictos cuando varias pantallas acceden a los datos al mismo tiempo. La base de datos se guarda en el dispositivo con el nombre `noteplusadmin.db`.

3. Carga inicial de datos (SeedCallback) Al crear la base de datos por primera vez, el `SeedCallback` inserta automáticamente todos los datos necesarios para que la app funcione desde el inicio:

- Las 12 nacionalidades predefinidas
- Los 3 roles del sistema (Administrador, Docente, Coordinador)
- La persona del administrador con sus datos básicos
- El usuario admin con su contraseña encriptada con SHA-256

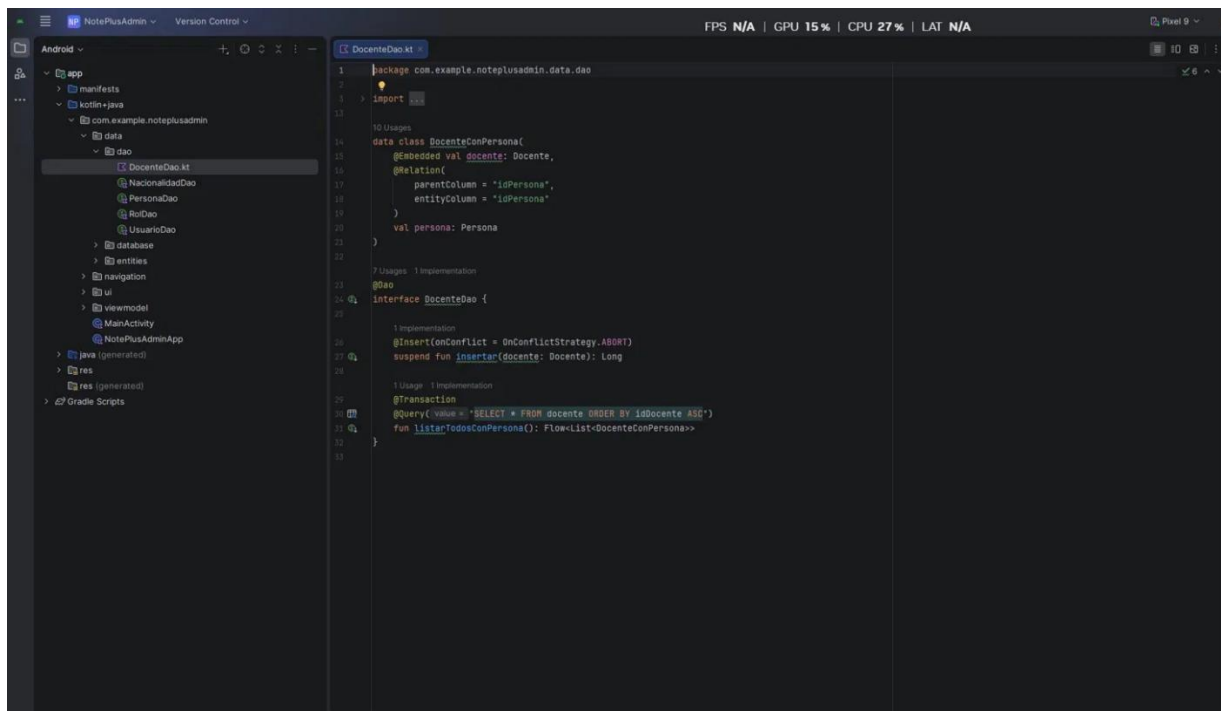
Este proceso se ejecuta una sola vez — la primera vez que se instala la app. En instalaciones posteriores la base de datos ya existe y el callback no se vuelve a ejecutar.

Acceso a la Base de Datos

Los DAOs son las interfaces que definen cómo la app se comunica con la base de datos. Room genera automáticamente el código necesario para ejecutar cada operación.

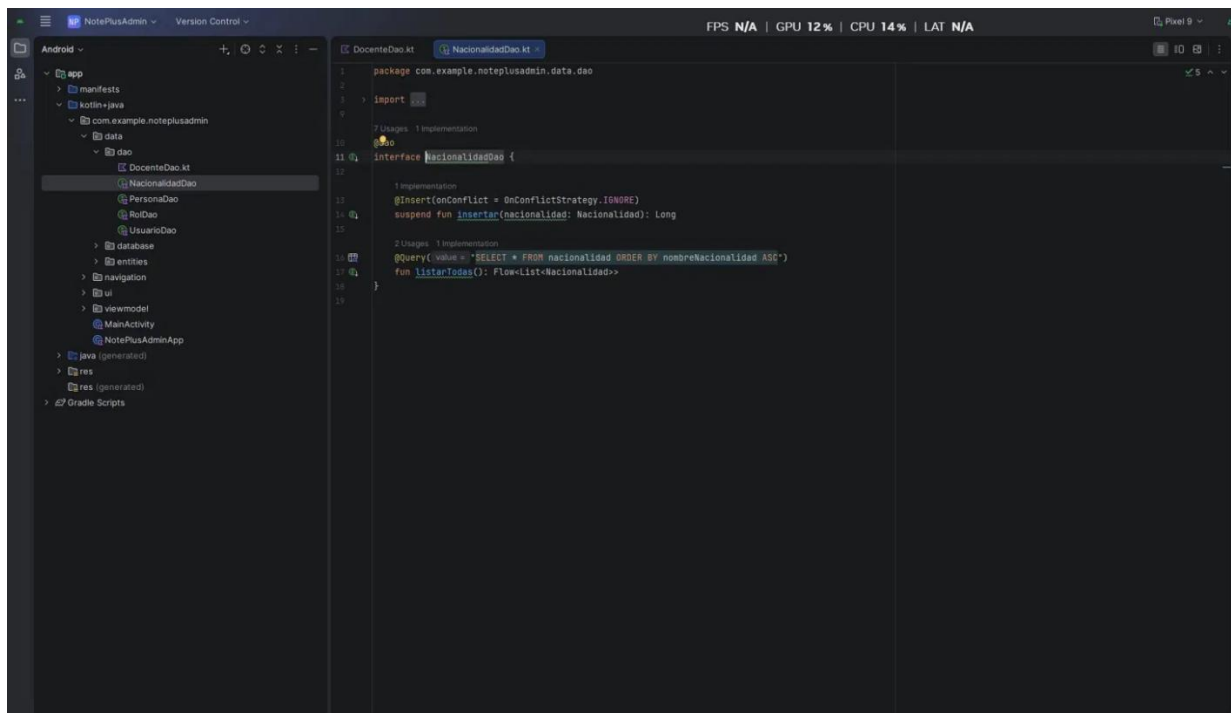
DocenteDao

Tiene dos operaciones: insertar un docente nuevo y listar todos los docentes junto con sus datos personales. Para el listado usa una clase especial DocenteConPersona que combina los datos de las tablas Docente y Persona en un solo resultado, evitando hacer dos consultas separadas.



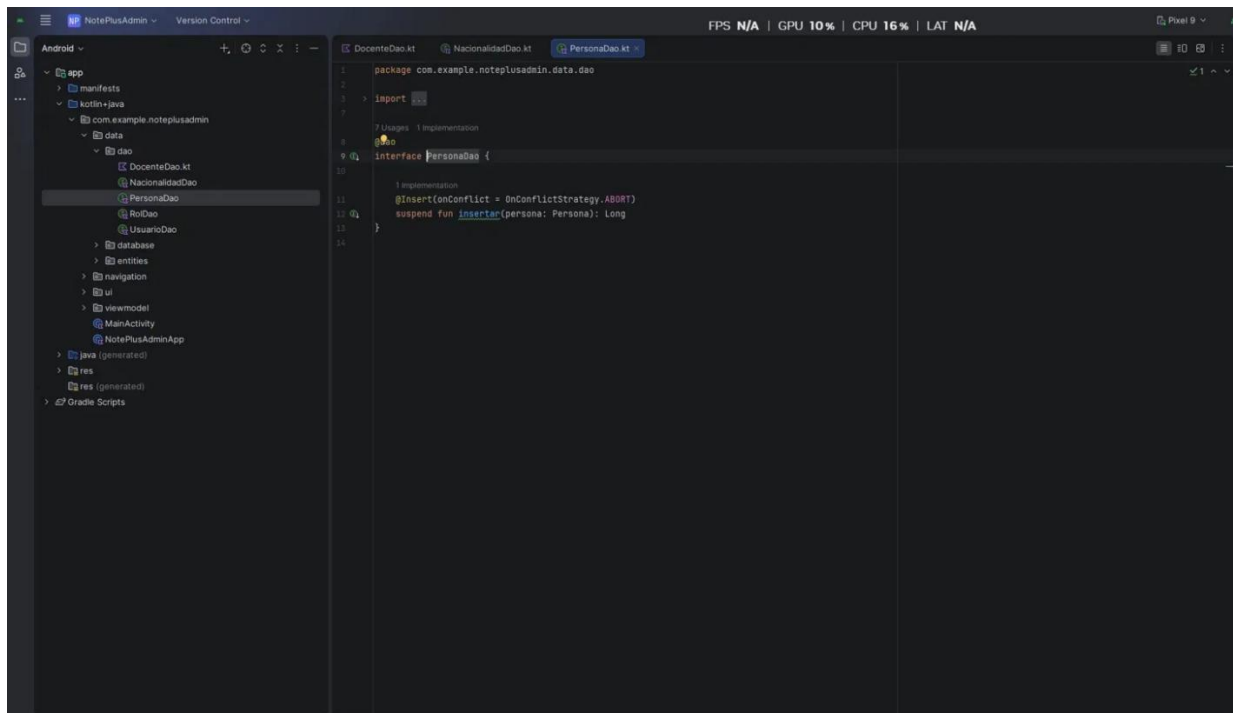
NacionalidadDao

Tiene dos operaciones: insertar una nueva nacionalidad y listar todas ordenadas alfabéticamente. Si se intenta insertar una nacionalidad duplicada, la operación se ignora silenciosamente (IGNORE) — así es como detecta duplicados y muestra el mensaje de error en la pantalla.



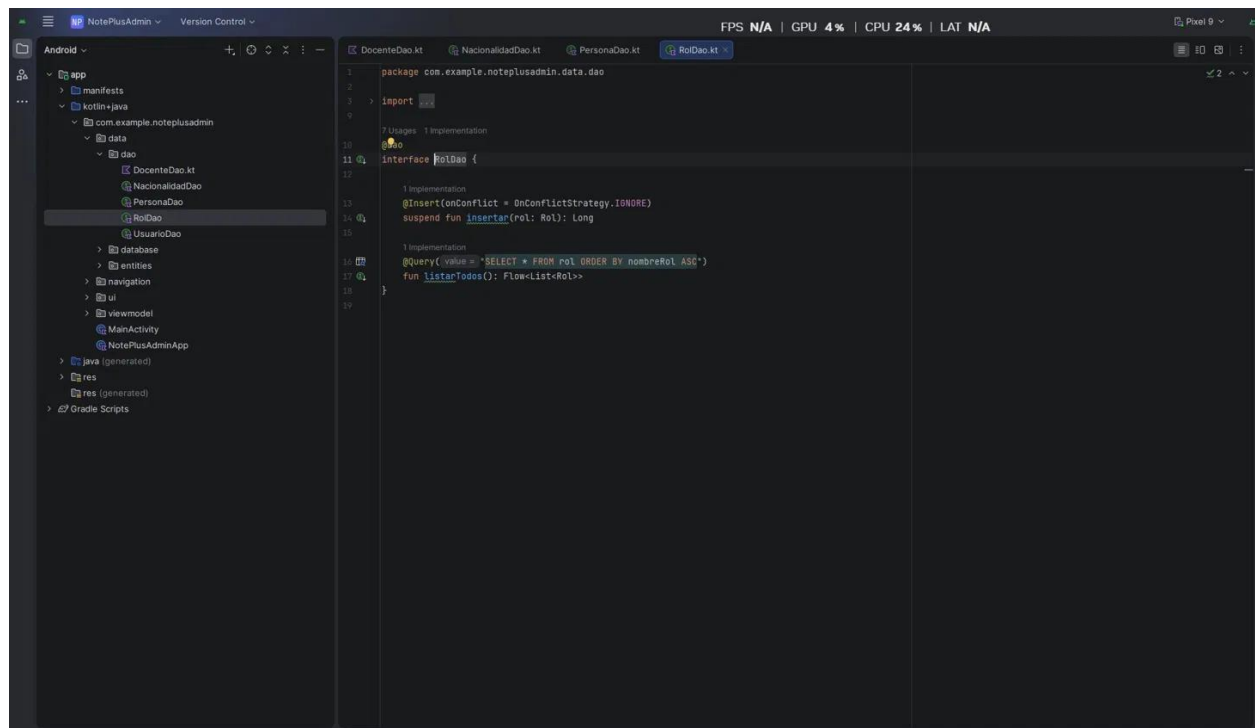
PersonaDao

Solo tiene una operación: insertar una persona nueva. Retorna el ID generado automáticamente, que luego se pasa a la pantalla de datos del docente para vincularlo correctamente. Si hay un error al insertar, la operación se cancela.



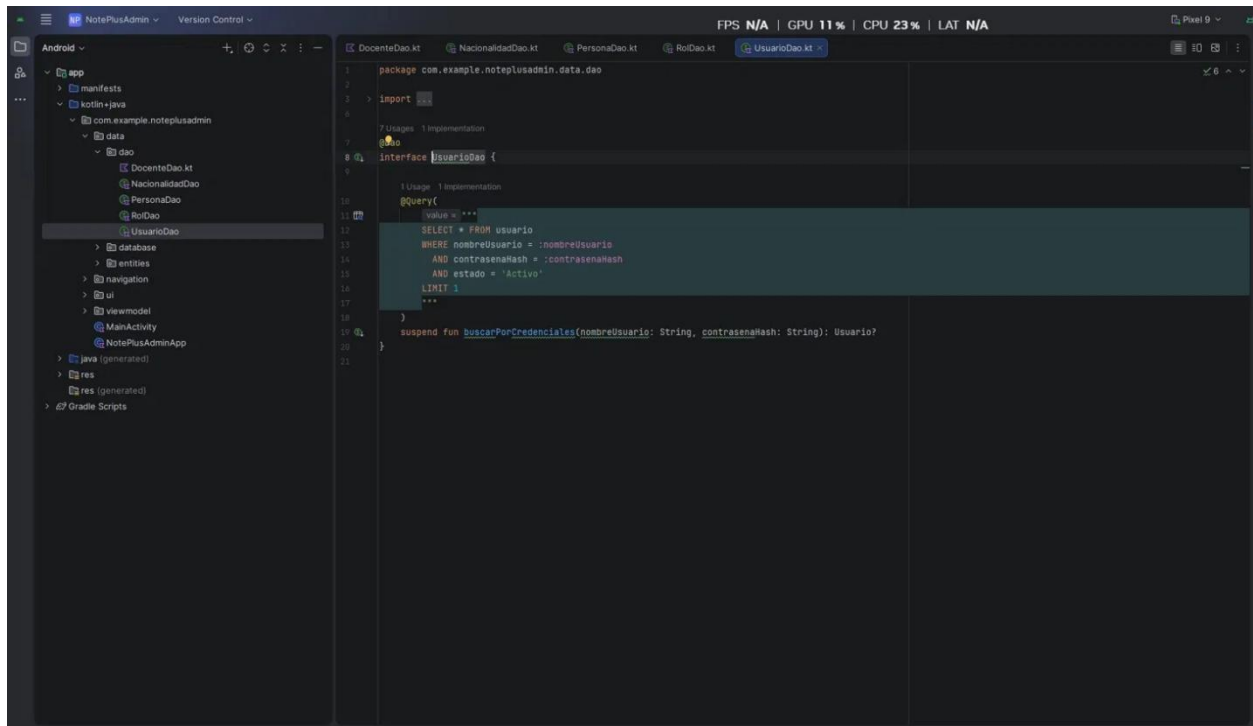
RolDao

Tiene dos operaciones: insertar roles y listarlos todos. Solo se usa internamente para cargar los roles al iniciar la base de datos por primera vez.



UsuarioDao

Tiene una sola operación — buscar un usuario por nombre de usuario, contraseña encriptada y estado activo simultáneamente. Si los tres valores coinciden con un registro en la base de datos, retorna el usuario y el login es exitoso. Si no hay coincidencia, retorna nulo y el login falla.

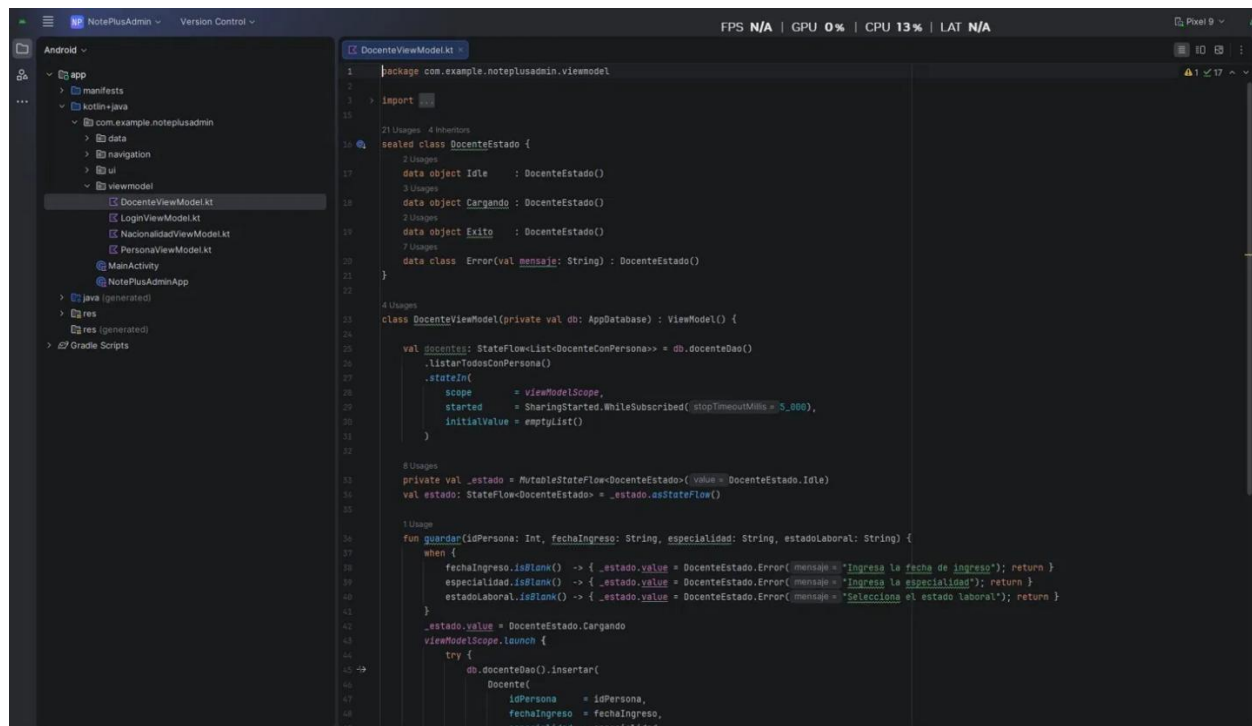


ViewModels

Los ViewModels son la capa intermedia entre la interfaz de usuario y la base de datos. Cada pantalla tiene su propio ViewModel que maneja el estado y las operaciones disponibles. Todos usan un patrón de estados (Idle, Cargando, Éxito, Error) para comunicarle a la pantalla qué está pasando en cada momento.

DocenteViewModel

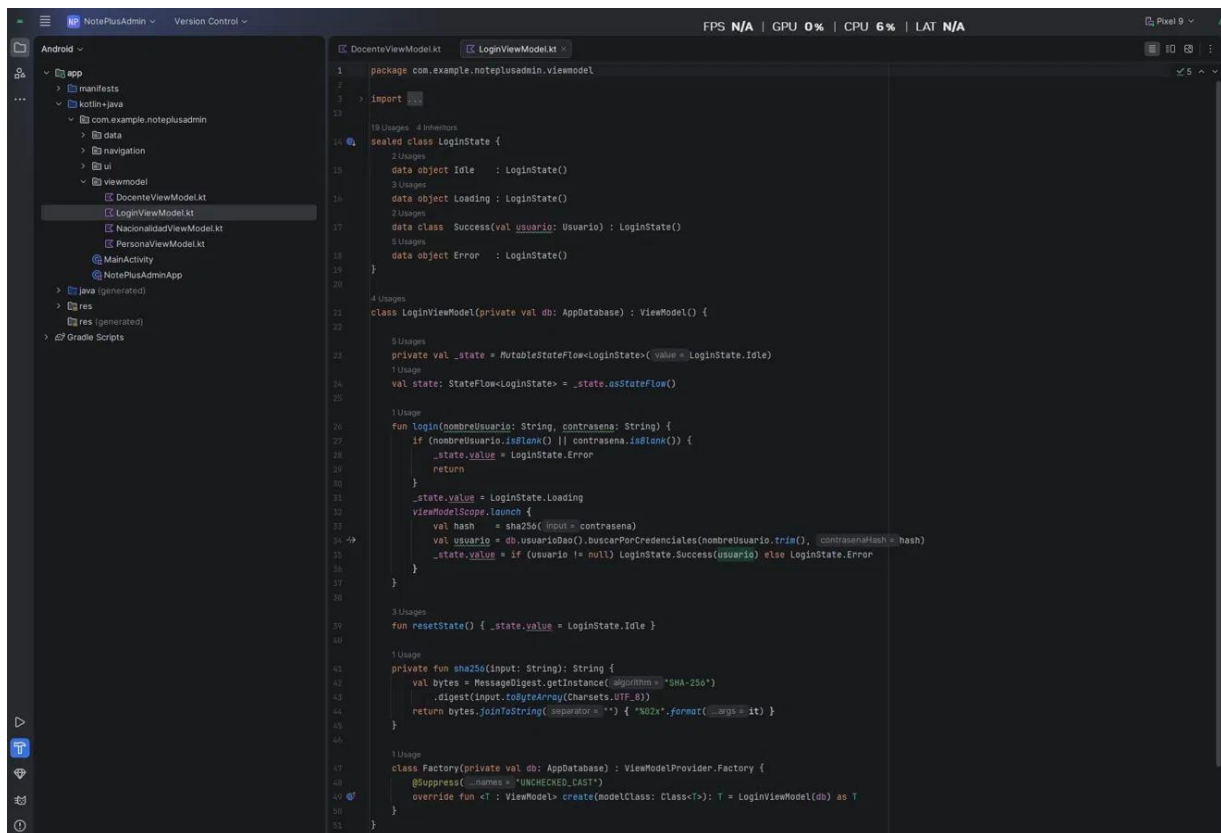
Maneja el registro de un nuevo docente. Recibe los datos laborales junto con el ID de la persona previamente creada, valida que ningún campo esté vacío y guarda el registro en la base de datos. También mantiene actualizada la lista de docentes en tiempo real.



```
1 package com.example.noteplusadmin.viewmodel
2
3 import androidx.lifecycle.ViewModel
4
5 sealed class DocenteEstado {
6     data object Idle : DocenteEstado()
7     data object Cargando : DocenteEstado()
8     data object Exito : DocenteEstado()
9     data class Error(val mensaje: String) : DocenteEstado()
10 }
11
12 class DocenteViewModel(private val db: AppDatabase) : ViewModel() {
13     val docentes: StateFlow<List<DocenteConPersona>> = db.docenteDao()
14         .listarTodosConPersona()
15         .stateIn(
16             scope = viewModelScope,
17             started = SharingStarted.WhileSubscribed(stopTimeoutMillis = 5_000),
18             initialValue = emptyList()
19         )
20
21     private val _estado = MutableStateFlow<DocenteEstado>(_estado = DocenteEstado.Idle)
22     val estado: StateFlow<DocenteEstado> = _estado.asStateFlow()
23
24     fun guardar(idPersona: Int, fechaIngreso: String, especialidad: String, estadoLaboral: String) {
25         when {
26             fechaIngreso.isBlank() -> { _estado.value = DocenteEstado.Error(mensaje = "Ingresar la fecha de ingreso"); return }
27             especialidad.isBlank() -> { _estado.value = DocenteEstado.Error(mensaje = "Ingresar la especialidad"); return }
28             estadoLaboral.isBlank() -> { _estado.value = DocenteEstado.Error(mensaje = "Seleccionar el estado laboral"); return }
29         }
30         _estado.value = DocenteEstado.Cargando
31         viewModelScope.launch {
32             try {
33                 db.docenteDao().insertar(
34                     Docente(
35                         idPersona = idPersona,
36                         fechaIngreso = fechaIngreso,
37                         especialidad = especialidad,
38                         estadoLaboral = estadoLaboral
39                     )
40                 )
41             } catch (e: Exception) {
42                 _estado.value = DocenteEstado.Error(mensaje = e.message ?: "Error desconocido")
43             }
44         }
45     }
46 }
```

LoginViewModel

Maneja el proceso de autenticación. Cuando el usuario pulsa "Iniciar Sesión", toma el nombre de usuario y la contraseña, convierte la contraseña a su forma encriptada y la compara contra la base de datos. Si hay coincidencia el estado cambia a Éxito y la app navega al Dashboard; si no, cambia a Error y muestra el mensaje correspondiente en la pantalla.



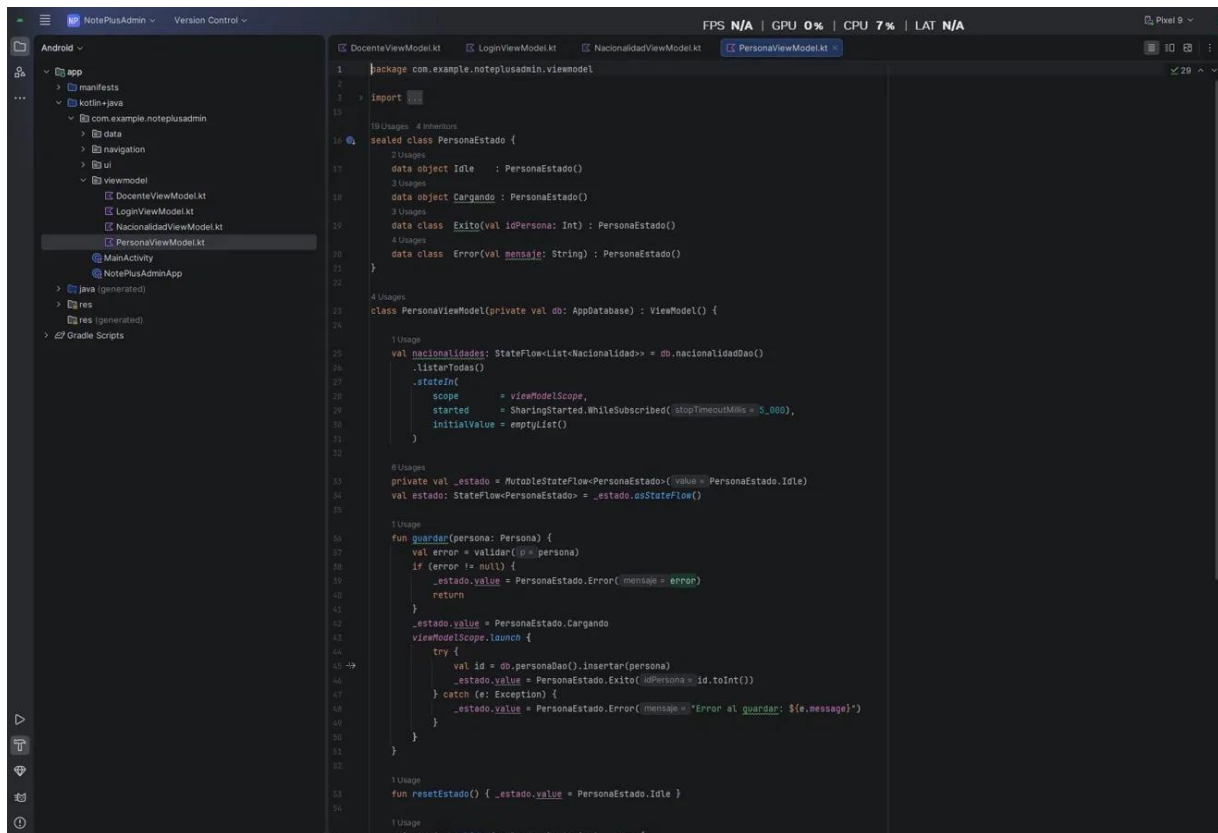
NacionalidadViewModel

Maneja la pantalla de nacionalidades. Mantiene la lista actualizada en tiempo real y gestiona la inserción de nuevas nacionalidades. Valida que el campo no esté vacío antes de guardar. Si la base de datos retorna un resultado negativo al insertar (porque el registro ya existe), cambia el estado a Error con el mensaje "Esa nacionalidad ya existe".

```
1 package com.example.noteplusadmin.viewmodel
2
3 import androidx.lifecycle.*
4
5 sealed class NacionaIdadGuardarEstado {
6     data object Idle : NacionaIdadGuardarEstado()
7     data object Exito : NacionaIdadGuardarEstado()
8     data class Error(val mensaje: String) : NacionaIdadGuardarEstado()
9 }
10
11 class NacionaIdadViewModel(private val db: AppDatabase) : ViewModel() {
12     val nacionalidades: StateFlow<List<Nacionalidad>> = db.nacionalidadDao().listarTodas()
13     .stateIn(
14         scope = viewModelScope,
15         started = SharingStarted.WhileSubscribed(stopTimeoutMillis = 5_000),
16         initialValue = emptyList()
17     )
18
19     private val _guardarEstado = MutableStateFlow<NacionaIdadGuardarEstado>(_guardarEstado = NacionaIdadGuardarEstado.Idle)
20     val guardarEstado: StateFlow<NacionaIdadGuardarEstado> = _guardarEstado.asStateFlow()
21
22     fun insertar(nombre: String) {
23         if (nombre.isBlank()) {
24             _guardarEstado.value = NacionaIdadGuardarEstado.Error(mensaje = "El nombre no puede estar vacío")
25             return
26         }
27         viewModelScope.launch {
28             val resultado = db.nacionalidadDao().insertar(
29                 Nacionalidad(nombreNacionalidad = nombre.trim())
30             )
31             _guardarEstado.value = if (resultado > 0) {
32                 NacionaIdadGuardarEstado.Exito
33             } else {
34                 NacionaIdadGuardarEstado.Error(mensaje = "Esa nacionalidad ya existe")
35             }
36         }
37     }
38
39     fun resetEstado() { _guardarEstado.value = NacionaIdadGuardarEstado.Idle }
40
41     class Factory(private val db: AppDatabase) : ViewModelProvider.Factory {
42         @Suppress("UNCHECKED_CAST")
43         override fun create(modelClass: Class<?>): ViewModel? {
44             return if (modelClass.isAssignableFrom(NacionaIdadViewModel::class)) {
45                 NacionaIdadViewModel(db) as ViewModel
46             } else null
47         }
48     }
49 }
```

PersonaViewModel

Maneja el formulario de datos personales. Valida todos los campos obligatorios antes de guardar. Si la validación pasa, inserta la persona en la base de datos y retorna el ID generado automáticamente en el estado de Éxito — ese ID es el que se pasa a la siguiente pantalla para vincular el docente. También carga las nacionalidades disponibles para el dropdown del formulario.



Sección 2

Para el desarrollo de la sección 2 se escogió el **Constraint Layout** sobre el mismo proyecto teniendo de esta forma dos pantallas de **Logín**.

Constraint Layout

Es un tipo de layout de Android que permite posicionar cada elemento de la pantalla de forma precisa, estableciendo restricciones o "anclas" entre los elementos y los bordes de la pantalla. A diferencia del Linear Layout donde los elementos simplemente se apilan en orden, en el Constraint Layout cada elemento sabe exactamente dónde debe estar porque tiene definidas sus relaciones con los demás.

Por ejemplo, se puede indicar que un botón siempre debe estar centrado horizontalmente en la pantalla, a una distancia fija del campo de texto que está encima, y siempre visible sin

importar el tamaño del dispositivo. Si se mueve el campo de texto, el botón se mueve con él automáticamente respetando la distancia definida.

Es el layout más flexible y recomendado por Google para pantallas complejas, ya que permite construir interfaces que se adaptan correctamente a diferentes tamaños de pantalla sin necesidad de crear múltiples versiones del diseño.

Codificación del Layout

La aplicación implementa dos versiones de la pantalla de login para demostrar el uso de diferentes tipos de layout en Android con Jetpack Compose.

Login con Linear Layout (Column).

La primera versión organiza todos los elementos en una columna vertical simple usando Column. El ícono, el título, los campos de usuario y contraseña, y el botón van uno debajo del otro en orden secuencial. Es la forma más directa de organizar una pantalla donde el contenido fluye naturalmente de arriba hacia abajo.

Login con Constraint Layout.

La segunda versión usa ConstraintLayout donde cada elemento tiene una posición definida mediante restricciones. Se puede ver claramente en el código cómo cada elemento se ancla a otro:

- El ícono se ancla a la parte superior de la pantalla con un margen de 64dp
- El título se ancla debajo del ícono con un margen de 8dp
- El subtítulo se ancla debajo del título con 4dp
- El campo de usuario se ancla debajo del subtítulo con 40dp
- El campo de contraseña se ancla debajo del campo de usuario con 12dp

- El botón se ancla a un "barrier" dinámico que sube o baja según si hay un mensaje de error visible
- El texto de credenciales y el botón de volver quedan anclados en la parte inferior de la pantalla

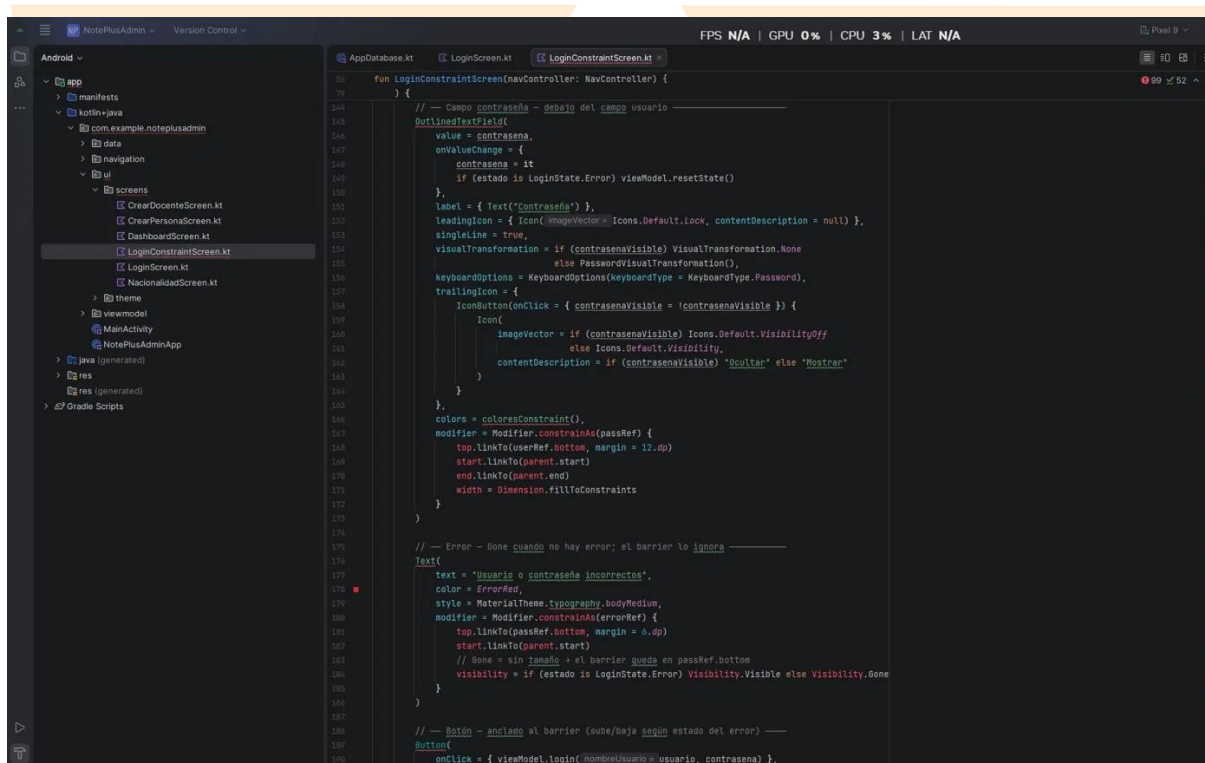
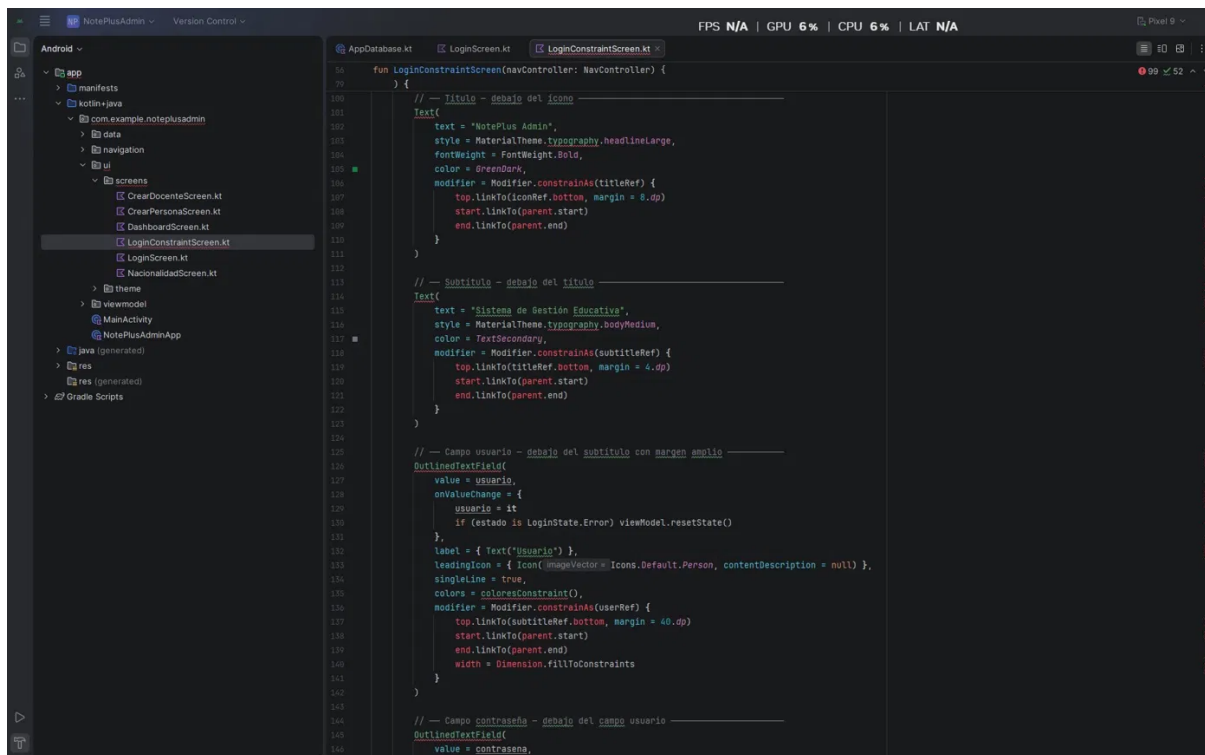
Integración entre ambas versiones

Las dos pantallas coexisten dentro del mismo proyecto. Desde la pantalla de Login original hay un botón **"Ver versión ConstraintLayout"** que navega a la segunda versión. Desde la versión Constraint hay un botón **"← Volver al login estándar"** para regresar. Ambas versiones usan el mismo LoginViewModel — la lógica de validación de credenciales es idéntica, solo cambia la forma en que se organizan visualmente los elementos en pantalla.

```

1 package com.example.noteplusadmin.ui.screens
2
3 import androidx.compose.foundation.layout.*
4
5 @Composable
6 fun LoginConstraintScreen(navController: NavController) {
7     val app = LocalContext.current.applicationContext as NotePlusAdminApp
8     val viewModel: LoginViewModel = viewModel(factory = LoginViewModel.Factory(app.database))
9     val estado by viewModel.state.collectAsState()
10
11     var usuario by remember { mutableStateOf("") }
12     var contrasena by remember { mutableStateOf("") }
13     var contrasenaVisible by remember { mutableStateOf(false) }
14
15     LaunchedEffect(key1 = estado) {
16         if (estado is LoginState.Success) {
17             navController.navigate(Screen.Dashboard.route) {
18                 popUpTo(Screen.LoginConstraint.route) { inclusive = true }
19             }
20             viewModel.resetState()
21         }
22     }
23
24     ConstraintLayout(
25         modifier = Modifier
26             .fillMaxSize()
27             .background(color = @white)
28             .padding(horizontal = 32.dp)
29     ) {
30         val (iconRef, titleRef, subtitleRef, userRef, passRef,
31             errorRef, btnRef, hintRef, backRef) = createRefs()
32
33         // Barrier dinámico: se mueve hacia abajo cuando aparece el error
34         val errorBarrier = createBottomBarrier(passRef, errorRef)
35
36         // Icono - anclado parte superior central
37         Icon(
38             imageVector = Icons.Default.School,
39             contentDescription = null,
40             tint = GreenDark,
41             modifier = Modifier
42                 .size(60.dp)
43                 .constraints(iconRef) {
44                     top.linkTo(parent.top, margin = 64.dp)
45                     start.linkTo(parent.start)
46                     end.linkTo(parent.end)
47                 }
48         )
    }
}

```

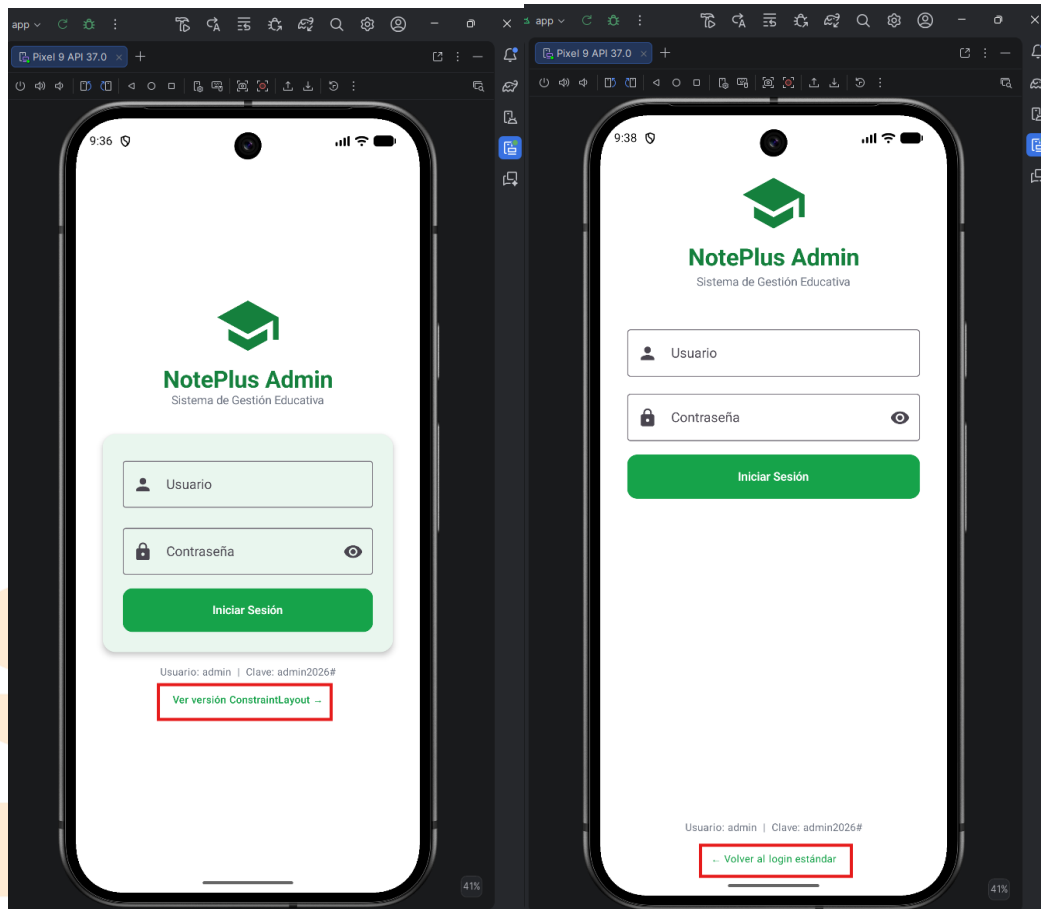
```
Android ~
├── app
│   ├── manifests
│   └── kotlin+java
│       ├── com.example.noteplusadmin
│       │   ├── data
│       │   ├── navigation
│       │   └── ui
│       │       ├── CrearDocenteScreen.kt
│       │       ├── CrearPersonaScreen.kt
│       │       ├── DashboardScreen.kt
│       │       ├── LoginConstraintScreen.kt
│       │       ├── LoginScreen.kt
│       │       └── NacionalidadScreen.kt
│       ├── theme
│       ├── viewmodel
│       ├── MainActivity
│       ├── NotePlusAdminApp
│       ├── java (generated)
│       ├── res
│       └── res (generated)
└── Gradle Scripts

AppDatabase.kt LoginScreen.kt LoginConstraintScreen.kt
fun LoginConstraintScreen(navController: NavController) {
} {
    // --- Botón - anclado al barrier (sube/baja según estado del error) ---
    Button(
        onClick = { viewModel.login(nombreUsuario = usuario, contraseña) },
        enabled = estado != LoginState.Loading,
        shape = RoundedCornerShape(size = 12.dp),
        colors = ButtonDefaults.buttonColors(containerColor = GreenPrimary),
        modifier = Modifier
            .height(32.dp)
            .constraints(0toRef) {
                top.linkTo(errorRef.barrier, margin = 16.dp)
                start.linkTo(parent.start)
                end.linkTo(parent.end)
                width = Dimension.fillToConstraints
            }
    ) {
        if (estado == LoginState.Loading) {
            CircularProgressIndicator(
                color = White,
                modifier = Modifier.size(22.dp),
                strokeWidth = 2.dp
            )
        } else {
            Text(
                text = "Iniciar Sesión",
                style = MaterialTheme.typography.labelLarge,
                fontWeight = FontWeight.SemiBold
            )
        }
    }
    // --- Texto de credenciales - anclado en la parte inferior ---
    Text(
        text = "Usuario: admin | Clave: admin2025a",
        style = MaterialTheme.typography.bodySmall,
        color = TextSecondary,
        modifier = Modifier.constraints(0toRef) {
            bottom.linkTo(backRef.top, margin = 4.dp)
            start.linkTo(parent.start)
            end.linkTo(parent.end)
        }
    )
    // --- Botón volver - anclado en la parte inferior ---
    TextButton(
        onClick = { navController.navigateUp() },
        modifier = Modifier.constraints(0toRef) {
            bottom.linkTo(parent.bottom, margin = 20.dp)
        }
    )
}
```

```
Android ~
├── app
│   ├── manifests
│   └── kotlin+java
│       ├── com.example.noteplusadmin
│       │   ├── data
│       │   ├── navigation
│       │   └── ui
│       │       ├── CrearDocenteScreen.kt
│       │       ├── CrearPersonaScreen.kt
│       │       ├── DashboardScreen.kt
│       │       ├── LoginConstraintScreen.kt
│       │       ├── LoginScreen.kt
│       │       └── NacionalidadScreen.kt
│       ├── theme
│       ├── viewmodel
│       ├── MainActivity
│       ├── NotePlusAdminApp
│       ├── java (generated)
│       ├── res
│       └── res (generated)
└── Gradle Scripts

AppDatabase.kt LoginScreen.kt LoginConstraintScreen.kt
fun LoginConstraintScreen(navController: NavController) {
} {
    // --- Texto de credenciales - anclado en la parte inferior ---
    Text(
        text = "Usuario: admin | Clave: admin2025a",
        style = MaterialTheme.typography.bodySmall,
        color = TextSecondary,
        modifier = Modifier.constraints(0toRef) {
            bottom.linkTo(backRef.top, margin = 4.dp)
            start.linkTo(parent.start)
            end.linkTo(parent.end)
        }
    )
    // --- Botón volver - anclado en la parte inferior ---
    TextButton(
        onClick = { navController.navigateUp() },
        modifier = Modifier.constraints(0toRef) {
            bottom.linkTo(parent.bottom, margin = 20.dp)
            start.linkTo(parent.start)
            end.linkTo(parent.end)
        }
    )
    Text(
        text = "Volver al login estándar",
        color = GreenPrimary,
        style = MaterialTheme.typography.bodySmall
    )
}

@Composable
private fun coloresConstraint() = OutlinedTextFieldDefaults.colors(
    focusedBorderColor = GreenPrimary,
    focusedLabelColor = GreenPrimary,
    focusedLeadingIconColor = GreenPrimary,
    focusedTrailingIconColor = GreenPrimary,
    cursorColor = GreenPrimary
)
```



Validación de Usuario con SQLite y Android Studio

La autenticación funciona de forma local, consultando directamente la base de datos SQLite del dispositivo. El proceso de validación ocurre en tres pasos:

Paso 1 Encriptación de la contraseña.

Cuando el usuario pulsa "Iniciar Sesión", el LoginViewModel toma la contraseña que escribió y la convierte a su forma encriptada. Esto garantiza que la contraseña nunca se compare en texto plano siempre se trabaja con su versión encriptada.

Paso 2 Consulta a la base de datos.

El UsuarioDao ejecuta una consulta SQL directamente sobre la tabla usuario de SQLite buscando un registro que cumpla tres condiciones simultáneamente:

- El nombreUsuario coincide exactamente con lo que escribió el usuario
- El contraseñaHash coincide con la contraseña encriptada
- El estado del usuario es 'Activo'

Si los tres valores coinciden, la consulta retorna el usuario. Si alguno no coincide, retorna nulo.

Paso 3 Respuesta de la app.

El LoginViewModel recibe el resultado de la consulta y actualiza el estado de la pantalla:

- Si retornó un usuario - estado cambia a LoginState.Success y la app navega al Dashboard
- Si retornó nulo - estado cambia a LoginState.Error y la pantalla muestra el mensaje

"Usuario o contraseña incorrectos" en color rojo

Este mismo proceso funciona igual en ambas versiones del login — tanto en la versión con Linear Layout como en la versión con Constraint Layout, ya que ambas comparten el mismo LoginViewModel.

